ERSIS

Enterprise Retail & Strategic Inventory System

Installation & Setup Guide

A Complete Beginner-Friendly Walkthrough

Recommended OS: Windows 10 / 11 • macOS 12+ • Ubuntu 22+

Table of Contents

1. Introduction

1.1 What is ERSIS?

ERSIS (Enterprise Retail & Strategic Inventory System) is a full-stack software platform designed to help retail businesses manage their inventory, process sales, track suppliers, and serve customers, all from a single, unified system.

The system is made up of three parts that work together:

- A web-based admin and cashier dashboard (runs in your browser)
- A customer-facing mobile app (runs on Android or iOS)
- An optional IoT barcode scanner integration (for hardware barcode scanners)

1.2 What Will You Achieve After Setup?

By the end of this guide, you will have the full ERSIS system running on your computer. Specifically, you will be able to:

- Access the web dashboard to manage products, categories, suppliers, and users
- Log in as an Admin, Cashier, or Customer using test accounts
- Run the mobile app on your smartphone using the Expo Go app
- View live API documentation in your browser
- (Optionally) connect a hardware barcode scanner

1.3 System Requirements

Requirement	Details
Operating System	Windows 10/11 (recommended), macOS 12+, or Ubuntu 22+
RAM	At least 8 GB (16 GB recommended for Docker)
Disk Space	At least 10 GB of free space
Internet	Required during setup (to download software and packages)
Processor	Any modern 64-bit CPU (Intel/AMD/Apple Silicon)

Note:
This guide primarily targets Windows users, but steps are provided for macOS/Linux where commands differ.

2. Required Software Installation

Before setting up the project itself, you need to install several tools on your computer. Think of these as the ingredients you need before cooking a meal you can't skip them.

This section walks you through each tool, explains what it does, and shows you exactly how to install it.

2.1 Git - Downloading the Project

Git is a tool that lets you download code from the internet (called "cloning"). Think of it as a special download manager for software projects.

Download link:

https://git-scm.com/install

Installation Steps

1. Open the link above in your browser and click the Windows download button.
2. Run the downloaded installer (e.g., Git-2.x.x-64-bit.exe).
3. Click Next through the setup. Leave all default options as they are; do not change anything unless you know what you're doing.
4. On the screen that asks about the default editor, you can select Visual Studio Code (we install that next).
5. Keep clicking Next until you see the Install button. Click Install.
6. Click Finish when done.

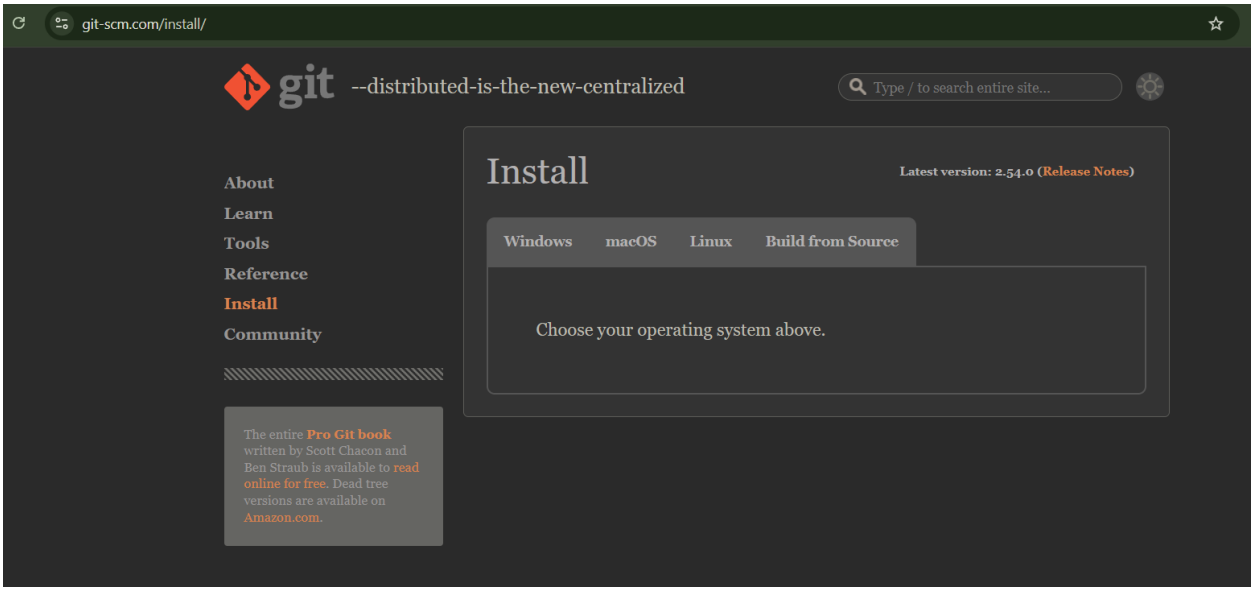

Fig: Git download page

Verify Git Installation

To check that Git installed correctly:

1. Press the Windows key, type cmd, and press Enter to open Command Prompt.
2. Type the following and press Enter:

```
git --version
```

3. You should see something like git version 2.53.0.windows.1

Success!
If you see a version number, Git is installed correctly. If you see 'git is not recognized', try restarting your computer and checking again.

2.2 Node.js - Running the Frontend

Node.js is a runtime environment that lets JavaScript code run outside of a browser. The ERSIS web frontend and mobile app are both built with JavaScript, so Node.js is needed to install and run them.

Download link (choose LTS version):
https://nodejs.org/en/download

Important!
You must install Node.js version 20 LTS (Long-Term Support). Do NOT install the latest version, as it may cause compatibility issues.

Installation Steps

1. Go to the link above and look for the section labeled '20.x.x LTS (Recommended for most users)'.
2. Download the Windows Installer (.msi) for 64-bit.
3. Run the installer. Click Next on every screen and keep all default settings.
4. On the screen 'Tools for Native Modules', you can leave the checkbox UNCHECKED unless instructed otherwise.
5. Click Install and wait for it to complete.
6. Click Finish.

Fig: Node.js website - LTS download button

Verify Node.js Installation

1. Open a new Command Prompt window.
2. Type both commands below and press Enter after each:

```
node --version
npm --version
```

3. You should see something like: v20.x.x and 10.x.x

2.3 Python 3.13 - Running the Backend

Python is a programming language. The ERSIS backend (the server-side part that manages your data) is written in Python.

Download link:

https://www.python.org/downloads/

Critical Step:

When installing Python, you MUST check the box that says 'Add Python to PATH' at the bottom of the first installation screen. If you miss this, Python commands won't work in your terminal.

Installation Steps

1. Go to python.org/downloads and click the button to download Python 3.13.x.
2. Run the installer (.exe file).
3. On the very first screen, check the box at the bottom: 'Add python.exe to PATH'. This is critical!
4. Click 'Install Now' (not the customized install option).
5. Wait for the installation to be completed. Click Close.

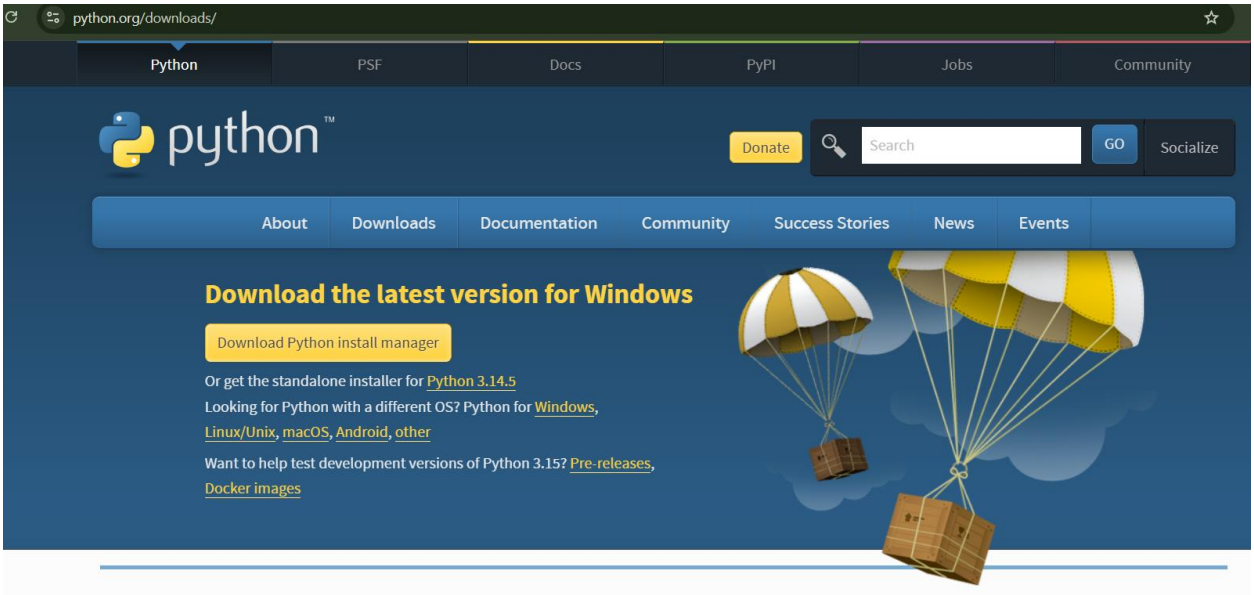

Fig: Python website - Download

Verify Python Installation

```
python --version
pip --version
```

You should see: Python 3.13.x and pip 24.x.x

Install uv (Python Package Manager)

ERSIS uses a faster Python package manager called uv. After Python is installed, open a Command Prompt and run:

```
pip install uv
```

Wait for it to finish. You should see 'Successfully installed uv'.

2.4 MySQL 8.0 - The Database

MySQL is the database where all ERSIS's data is stored, including products, users, transactions, and more. Think of it as a very organized filing cabinet for the application's data.

Download MySQL Community Server (free):

https://dev.mysql.com/downloads/mysql/

Installation Steps

1. On the download page, select Windows as your operating system and download the MySQL Installer.
2. Run the installer. When asked for setup type, choose 'Developer Default' or 'Server Only'.
3. Click Next, then Execute to download and install the components.
4. When you reach the 'Accounts and Roles' screen, set a root password. Write this password down, you will need it later!
5. Keep all other settings at their defaults and click Next until the installation is complete.
6. Click Finish to complete the setup.

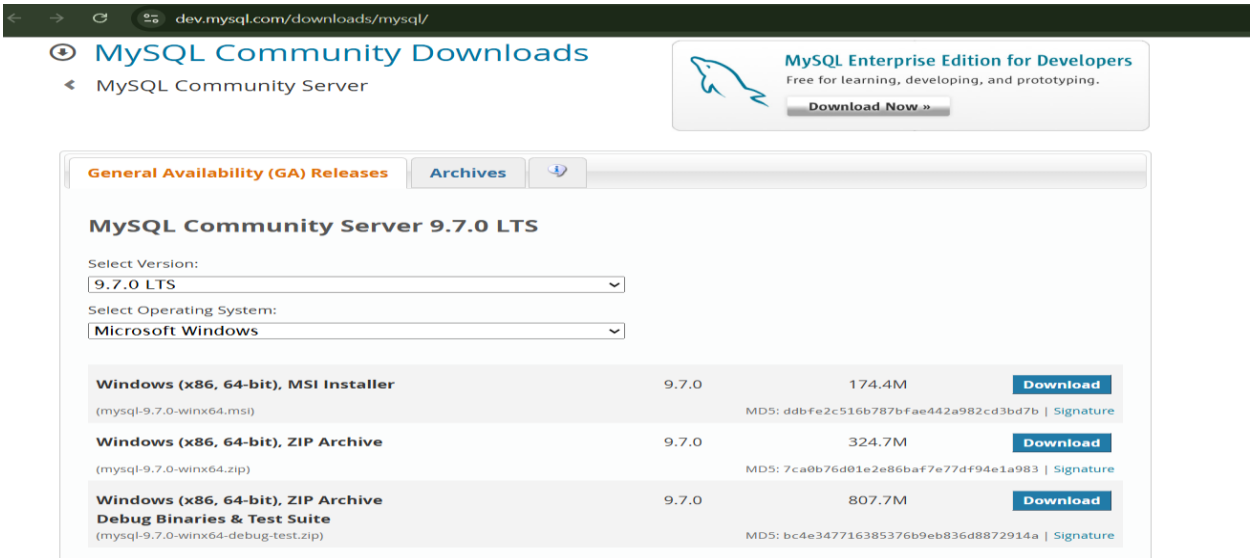

Fig: MySQL Download website

Remember Your Password!

The MySQL root password you set here is needed to connect ERSIS to the database. Store it somewhere safe (like a text file on your desktop labeled 'MySQL Password').

Install MySQL Workbench (Optional but Recommended)

MySQL Workbench is a visual tool that lets you see and manage your database. It's very helpful for beginners. You can download it from:

https://dev.mysql.com/downloads/workbench/

2.5 Visual Studio Code — Your Code Editor

Visual Studio Code (VS Code) is a free code editor made by Microsoft. You'll use it to open, view, and manage the project files and to run commands in the built-in terminal.

Download link:

https://code.visualstudio.com/

Installation Steps

7. Go to code.visualstudio.com and click the blue Download for Windows button.
8. Run the installer (.exe file).
9. Accept the license agreement.
10. On the 'Select Additional Tasks' screen, check the following boxes:
    - Add 'Open with Code' action to Windows Explorer file context menu
    - Add 'Open with Code' action to Windows Explorer directory context menu
    - Add to PATH (important!)
11. Click Next and then Install.
12. Click Finish. VS Code will open automatically.

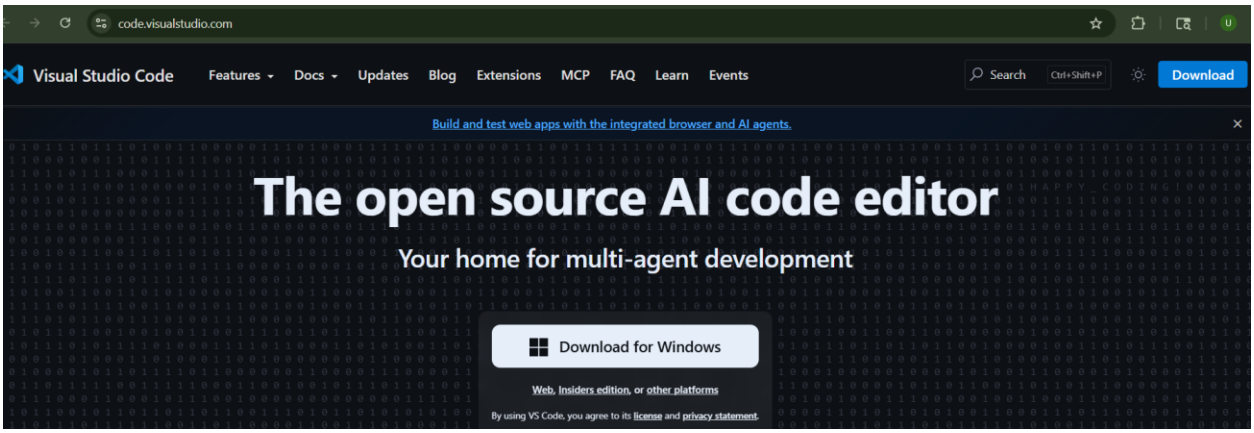

Fig: VS Code website — download button

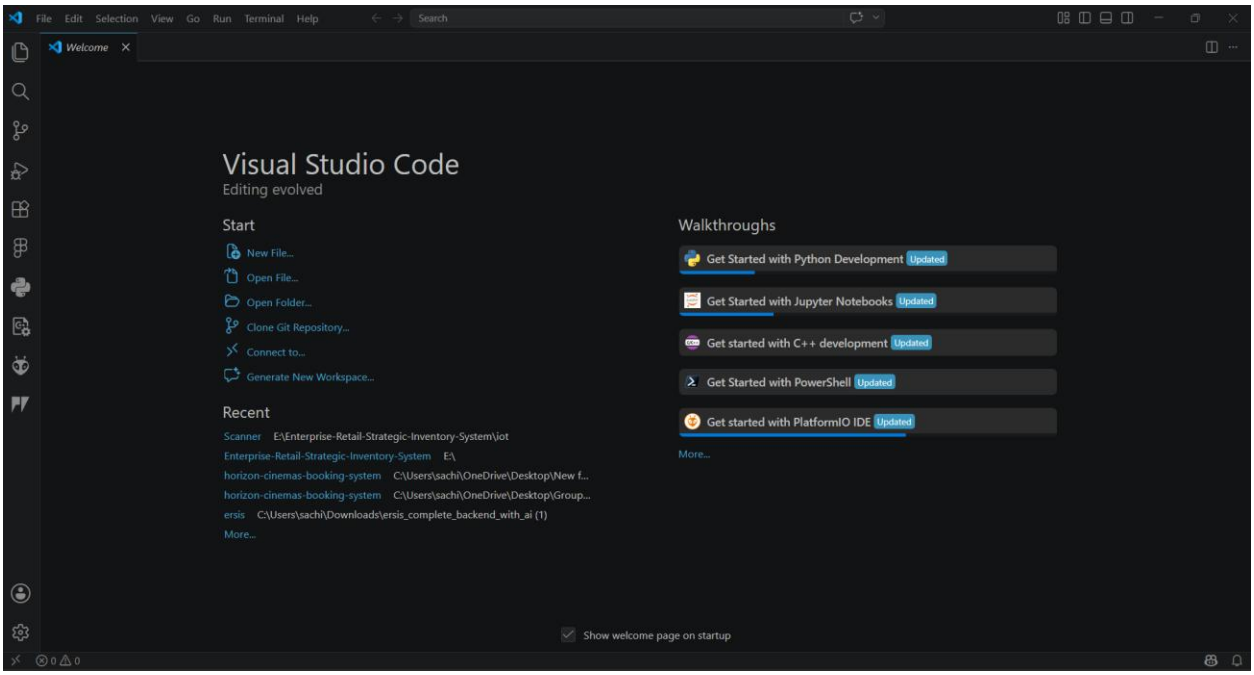

Fig: VS Code open for the first time

2.6 Docker Desktop — Optional

Docker is a tool that runs the entire project in a controlled environment called a 'container'. Using Docker means you don't need to manually install Python, Node.js, or MySQL. Docker handles everything automatically.

> **Skip for Now**
> If you are following the Manual Installation method (Section 4), you do not need to install Docker right now. Come back to this section when you're ready to try the Docker method (Section 6).

Download link:

https://www.docker.com/products/docker-desktop/

Installation Steps

1. Go to the link above and click 'Download Docker Desktop for Windows'.
2. Run the installer. Leave all settings at their defaults.
3. When prompted to 'Use WSL 2 instead of Hyper-V', keep it checked (this is the recommended option).
4. Click OK and let the installation complete. Your computer may restart.
5. After restarting, Docker Desktop will start automatically. Look for the whale icon in your system tray (bottom-right of the taskbar).

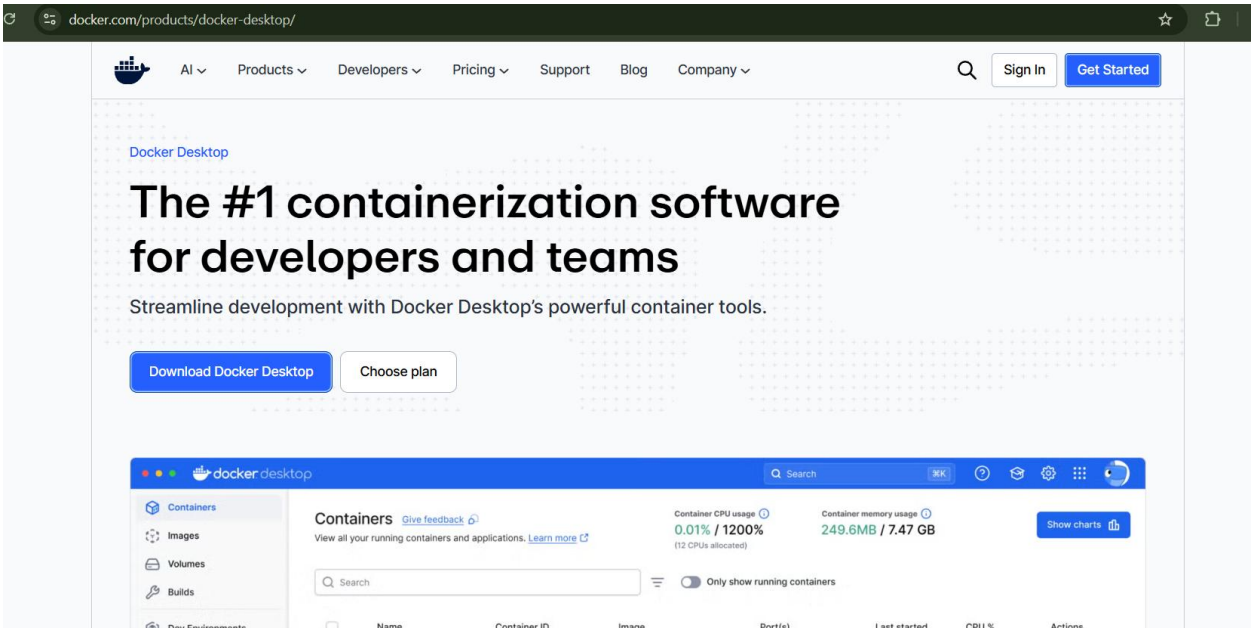

Fig: Docker Desktop website download page

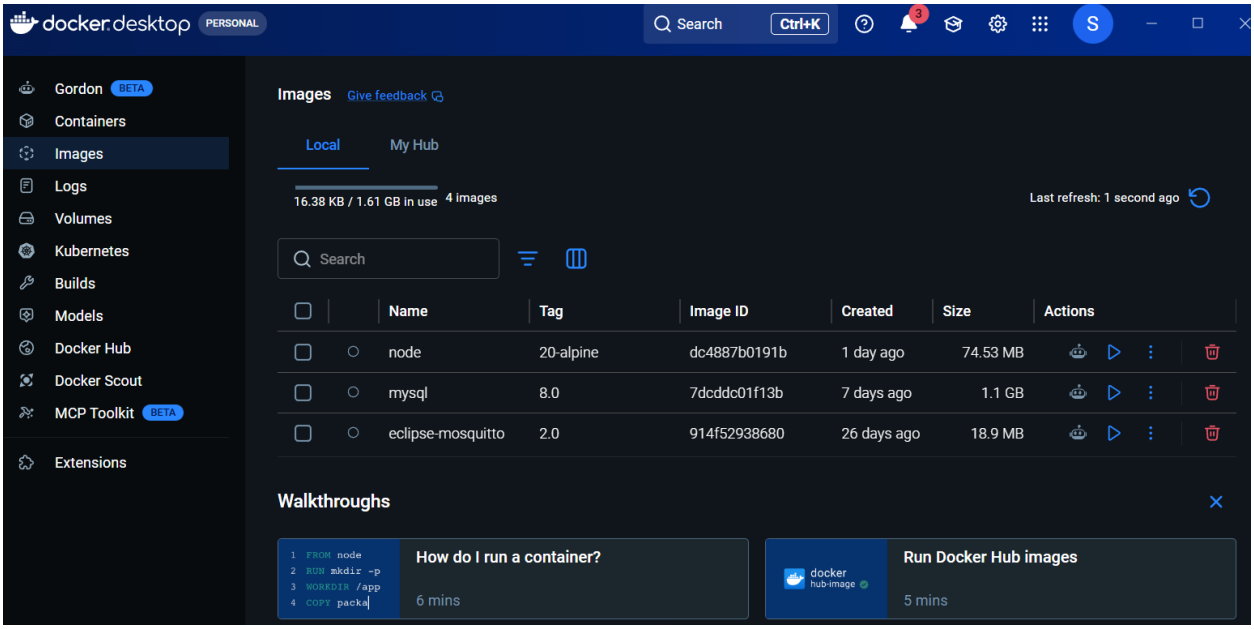

Fig: Docker Desktop running — whale icon in taskbar

WSL 2

WSL 2 (Windows Subsystem for Linux 2) is a feature built into Windows that Docker uses to run Linux-based containers efficiently. You don't need to do anything extra — Docker sets it up automatically.

3. VS Code Setup & Project Opening

Now that VS Code is installed, let's set it up properly and open the ERSIS project inside it.

3.1 Downloading the Project

The code is provided in the “Implementation and Code” folder. If you receive the project as a ZIP file, follow the steps below to extract it. If it’s not in a ZIP file, skip the part about extracting a ZIP file. If you will clone it using Git, skip Section 3.2.

Extracting a ZIP File

1. Locate the ZIP file (e.g., on your Desktop or in your Downloads folder).
2. Right-click the ZIP file.
3. Click 'Extract All...'
4. Choose a simple location like C:\Projects\ or your Desktop. Avoid paths with spaces or special characters (e.g., do not use 'My Documents').
5. Click Extract.
6. A new folder will appear, this is your project folder.

Avoid OneDrive / Dropbox Folders

Do not extract the project into a OneDrive or Dropbox folder. Sync tools can interfere with development and cause strange errors.

3.2 Opening the Project in VS Code

- Open VS Code.
- Click File in the top menu bar.
- Click Open Folder...
- Navigate to and select your project folder (the one that contains folders like backend, frontend-web, etc.).
- Click Select Folder.
- VS Code may ask, 'Do you trust the authors of the files in this folder?' Click Yes, I trust the authors.

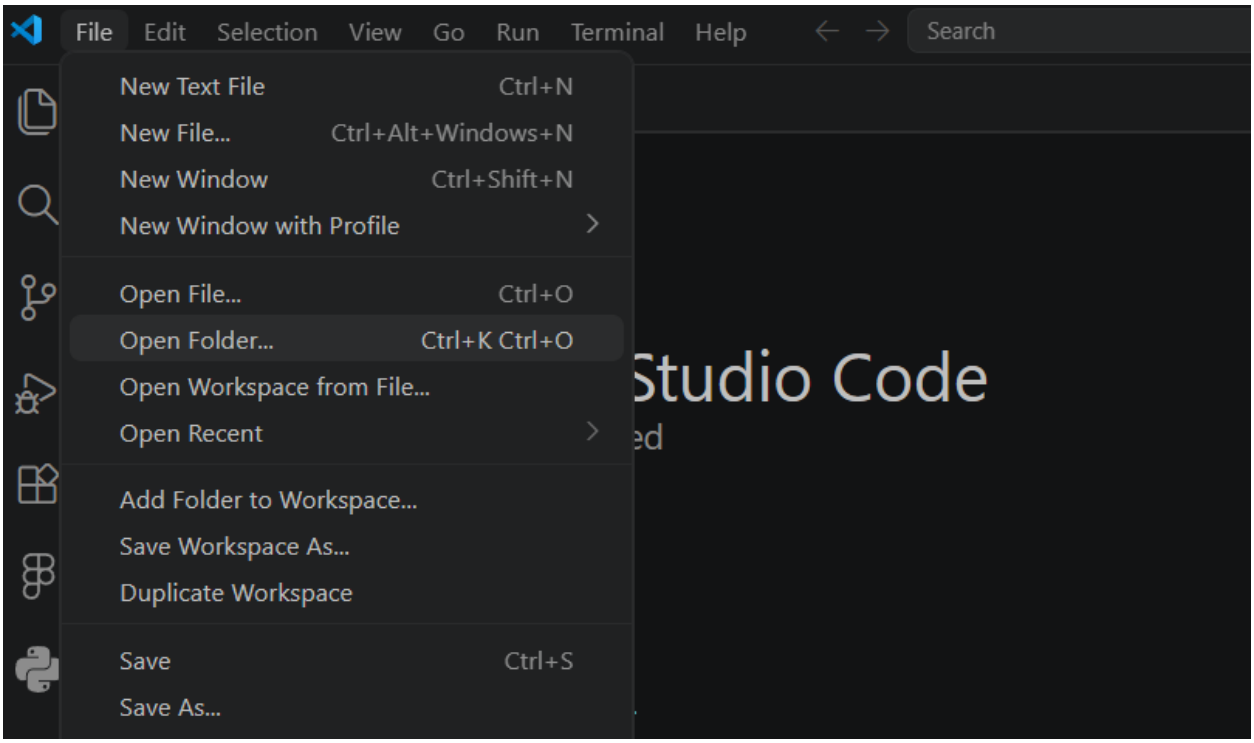

Fig: VS Code — File > Open Folder menu

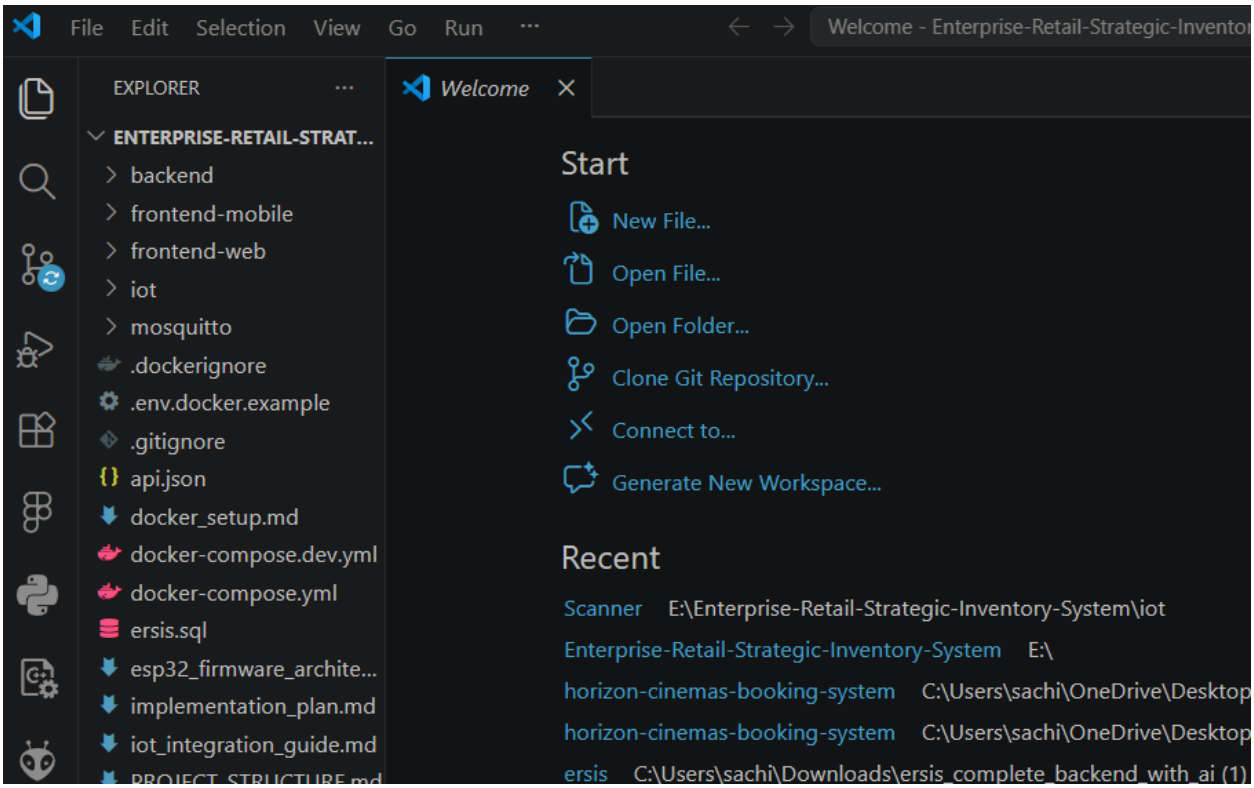

Fig: VS Code with ERSIS project folder open in sidebar

3.3 Recommended VS Code Extensions

Extensions add extra features to VS Code. Install these to make working on ERSIS easier:

Extension Name	What It Does
Python (by Microsoft)	Enables Python syntax highlighting and code suggestions
Pylance	Makes Python code smarter with better auto-complete
ESLint	Checks your JavaScript/React code for errors automatically
Prettier - Code Formatter	Automatically formats your code to look clean
GitLens	Shows who changed what in Git history
MySQL (by cweijan)	Lets you manage your MySQL database from inside VS Code

How to Install Extensions

1. Click the Extensions icon in the left sidebar (it looks like four squares).
2. In the search bar at the top, type the name of the extension.
3. Find the correct extension in the results and click the blue Install button.
4. Repeat each extension listed above.

Fig: VS Code Extensions panel — searching for 'Python'

3.4 Opening the Terminal in VS Code

Most setup steps require you to type commands into a terminal. VS Code has a built-in terminal, which you'll use throughout the guide.

How to Open the Terminal

1. In VS Code, click Terminal in the top menu bar.
2. Click New Terminal.
3. A panel will open at the bottom of the screen. This is your terminal.

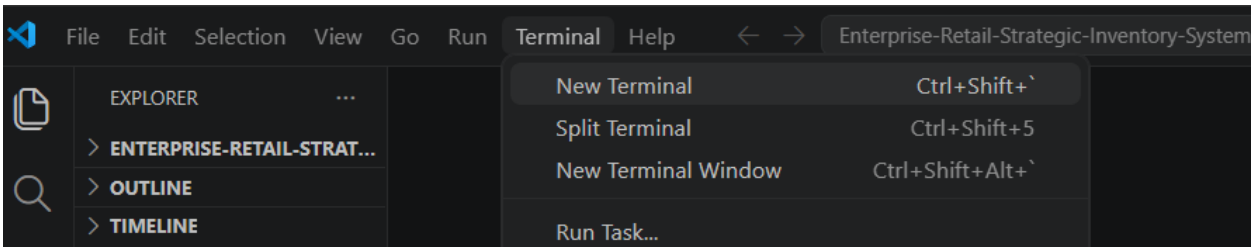

Fig: VS Code — Terminal > New Terminal menu option

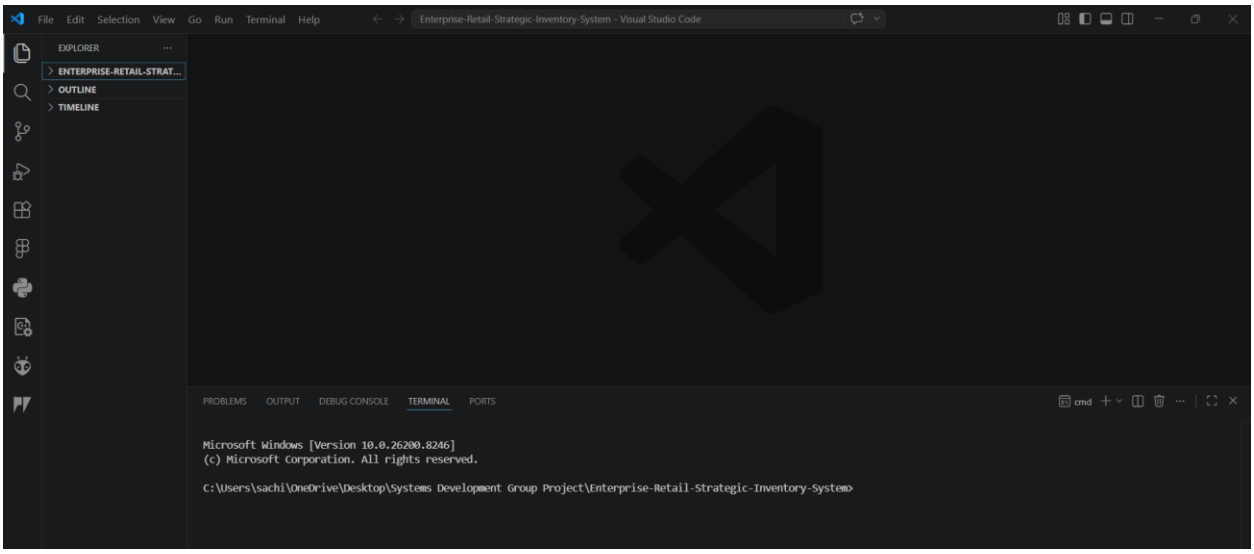

Fig: VS Code terminal panel open at the bottom

Windows Tip

If the terminal shows 'PowerShell', that's fine. All commands in this guide work in PowerShell. If you prefer Command Prompt, click the '+' dropdown arrow next to the terminal tabs and select 'Command Prompt'.

4. Project Setup - Manual Method (Primary)

This is the primary method for setting up ERSIS. Follow these steps in order. Do not skip any steps.

Estimated Time
30–45 minutes on a typical internet connection.

Step 1 - Clone (Download) the Project

Open the VS Code terminal (or any Command Prompt) and run the following two commands. These download the project from the internet and enter the project folder.

Command 1 — Download the project:

```
git clone https://github.com/sachincode11/Enterprise-Retail-Strategic-Inventory-System.git
```

Command 2 — Enter the project folder:

```
cd Enterprise-Retail-Strategic-Inventory-System
```

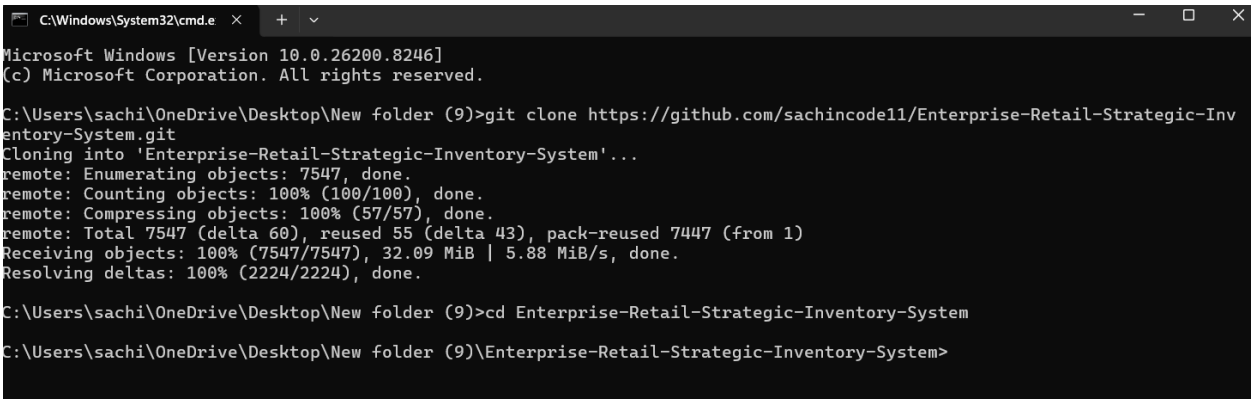

Fig: Terminal after git clone completes — folder created

What is 'cd'?
'cd' stands for 'change directory'. It moves the terminal into a specific folder; the same way you'd double-click a folder in File Explorer.

Step 2 - Set Up the MySQL Database

Before starting the backend, you need to create an empty database called 'ersis' in MySQL. ERSIS will store all its data inside this database.

Option A: Using MySQL Workbench (Recommended for Beginners)

1. Open MySQL Workbench from your Start Menu.
2. Click on the local MySQL connection (usually called 'Local instance MySQL80').
3. Enter your MySQL root password when prompted.
4. In the query window at the top, type:

```
CREATE DATABASE ersis;
```

5. Click the lightning bolt icon (or press Ctrl+Enter) to run the command.
6. You should see 'Action Output: 1 row(s) affected' at the bottom.

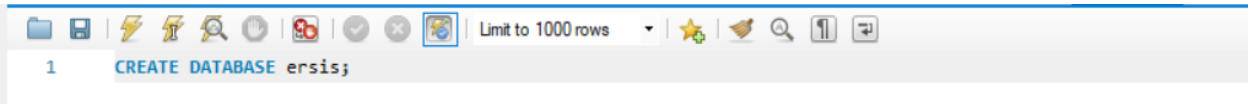

Fig: MySQL Workbench — typing CREATE DATABASE ersis

Option B: Using the Command Line

Open Command Prompt and type:

```
mysql -u root -p
```

Enter your MySQL password. Then type:

```
CREATE DATABASE ersis;
EXIT;
```

Step 3 - Configure the Backend

The backend (server) needs a configuration file that tells it your database password, email settings, and other private details. This file is called .env (pronounced 'dot env').

Step 3a — Navigate to the backend folder

In the terminal, type:

```
cd backend
```

Step 3b — Create the .env file

On Windows:

```
copy .env.example .env
```

On macOS or Linux:

```
cp .env.example .env
```

What is .env?

.env files store private settings (like passwords) that should not be shared. The project comes with a .env.example file as a template — we copy it to .env and then fill in our actual values.

Step 3c — Edit the .env file

Now you need to open the .env file in VS Code and fill in your details. In the VS Code file explorer on the left, find the backend folder, then click .env to open it.

Fill in each line as shown below. Replace the placeholder values with your actual information:

```
# Database connection
DATABASE_URL=mysql+pymysql://root:YOUR_MYSQL_PASSWORD@localhost/ersis

# Groq AI (for the AI assistant feature)
GROQ_API_KEY=gsk_...your_key_here...
```

```
# JWT secret (make up a long random string)
JWT_SECRET_KEY=my-super-long-random-secret-string-change-me

# Email (for OTP login emails)
SMTP_USER=your-email@gmail.com
SMTP_PASSWORD=your-gmail-app-password
EMAIL_FROM=your-email@gmail.com
```

Setting	What to Enter
YOUR_MYSQL_PASSWORD	The MySQL root password you set during MySQL installation
GROQ_API_KEY	Get a free API key at console.groq.com (sign up is free)
JWT_SECRET_KEY	Any long random string at least 32 characters. E.g.: 'mySecretKey2024!ChangeThisInProduction'
SMTP_USER	Your Gmail address (used to send OTP verification emails)
SMTP_PASSWORD	A Gmail App Password, NOT your regular Gmail password (see tip below)

Gmail App Password

A Gmail App Password is a special 16-character password that lets apps access Gmail. To generate one: Go to myaccount.google.com → Security → 2-Step Verification → App passwords. Select 'Mail' and 'Windows Computer', then generate. Copy the 16-character code into SMTP_PASSWORD.

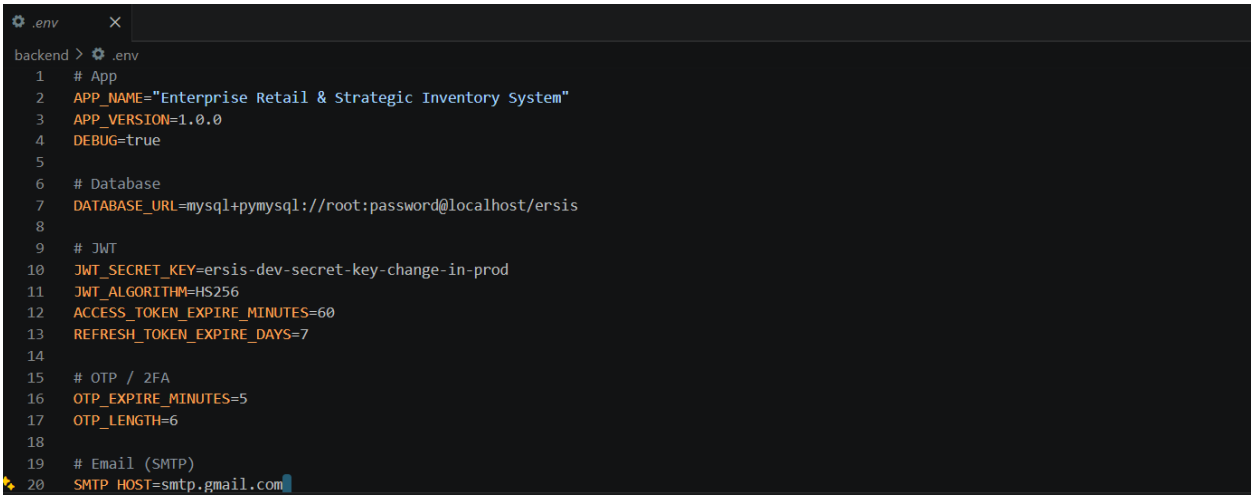

Fig: backend/.env file open in VS Code with values filled in

Step 4 - Install Backend Dependencies

Dependencies are extra packages of code that ERSIS's backend needs to work. We use a tool called 'uv' to install them quickly.

Make sure your terminal is still inside the backend folder. Then run these commands one at a time:

Create a virtual environment (an isolated space for Python packages):

```
uv venv
```

Install all required packages:

```
uv sync
```

What is a virtual environment?

A virtual environment is a self-contained folder that holds all the Python packages for this project. This keeps the project's packages separate from your system-wide Python, preventing conflicts.

Activate the virtual environment:

On Windows:

```
.venv\Scripts\activate
```

On macOS/Linux:

```
source .venv/bin/activate
```

You will see (ersis) or (.venv) appear at the start of the terminal prompt. This means the virtual environment is active.

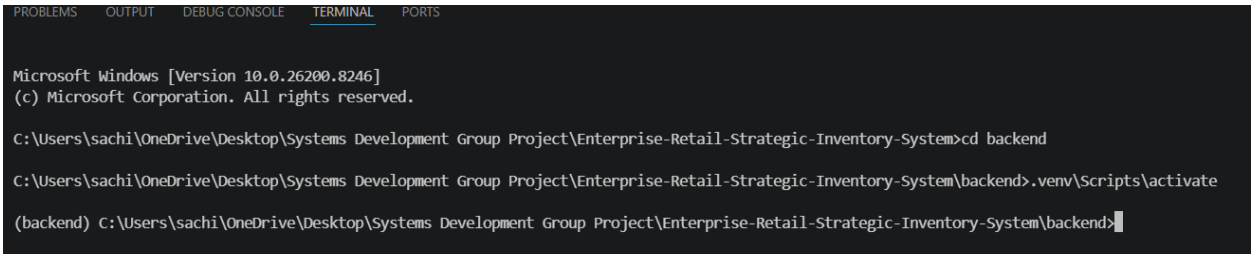

Fig: Terminal showing (backend) prefix after activation

Step 5 - Seed (Populate) the Database

The database is currently empty. The 'seed' command fills it with sample data, users, products, categories, transactions, and AI training data. This only needs to be done once.

Make sure the virtual environment is active (you see .venv in the prompt), then run:

```
python seed.py
```

This will take 1–3 minutes. You will see text scrolling in the terminal as data is inserted. Wait until you see 'Seeding complete' or a similar success message.

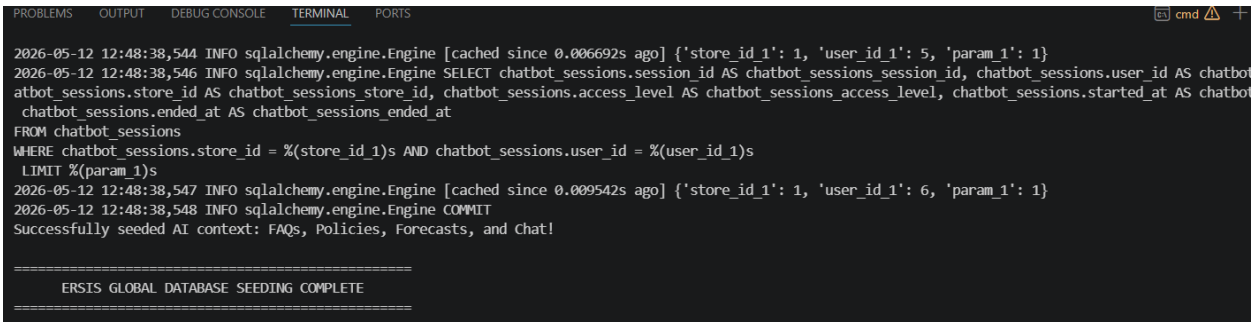

Fig: Terminal showing seed.py running and completing successfully

After seeding, you can log in with these test accounts:

Role	Email	Password
Admin	admin_seed@store.np	Password@123
Cashier	cashier_seed1@store.np	Password@123
Customer	customer_seed1@store.np	Password@123

Step 6 - Start the Backend Server

Now start the backend API server. This is the engine of the application — it handles all data requests from the web frontend and mobile app.

Run this command (keep the virtual environment active):

```
python -m uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

What does this command do?

--reload: Automatically restarts the server when you save changes to the code.
--host 0.0.0.0: Makes the server accessible from your mobile phone on the same network.
--port 8000: The server runs on port 8000 of your computer.

You should see output similar to:

```
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO:     Started reloader process
```

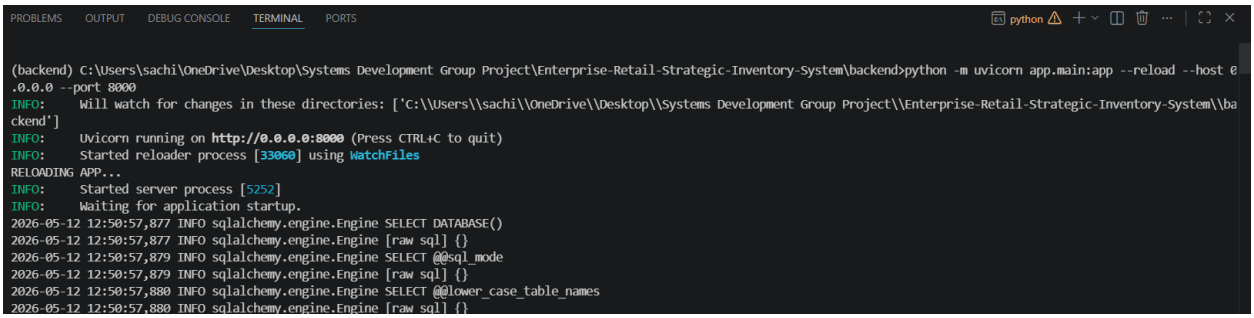

Fig: Terminal showing Uvicorn server running on port 8000

Keep This Terminal Open

Do NOT close this terminal window. The backend server runs as long as this terminal is open. To stop it, press Ctrl+C.

Test that the backend is working by opening this link in your browser:

http://127.0.0.1:8000/docs

You should see the Swagger API documentation page.

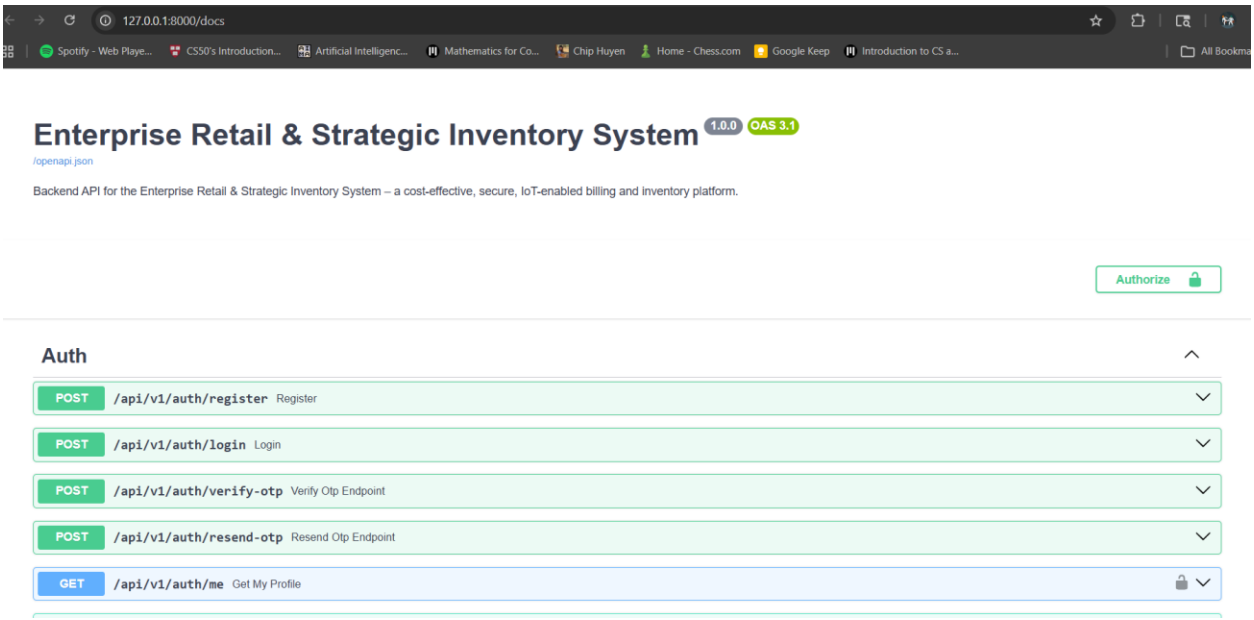

Fig: Swagger API docs page at http://127.0.0.1:8000/docs

Step 7 - Set Up the Web Frontend

The web frontend is the browser-based user interface, the dashboard that admins and cashiers use. It is built with React and Vite.

Open a NEW terminal window in VS Code (click the + icon in the terminal panel). Then navigate to the frontend-web folder from the project root:

```
cd frontend-web
```

Create the frontend's .env file:

```
copy .env.example .env
```

Install all dependencies (this may take 2–5 minutes):

```
npm install
```

Start the development server:

```
npm run dev
```

After a few seconds, you'll see:

```
VITE v5.x.x  ready in 500 ms
➜ Local:   http://localhost:5173/
```

Fig: Vite dev server running — showing localhost:5173 URL

Open your browser and go to:

http://localhost:5173

The ERSIS login page should appear!

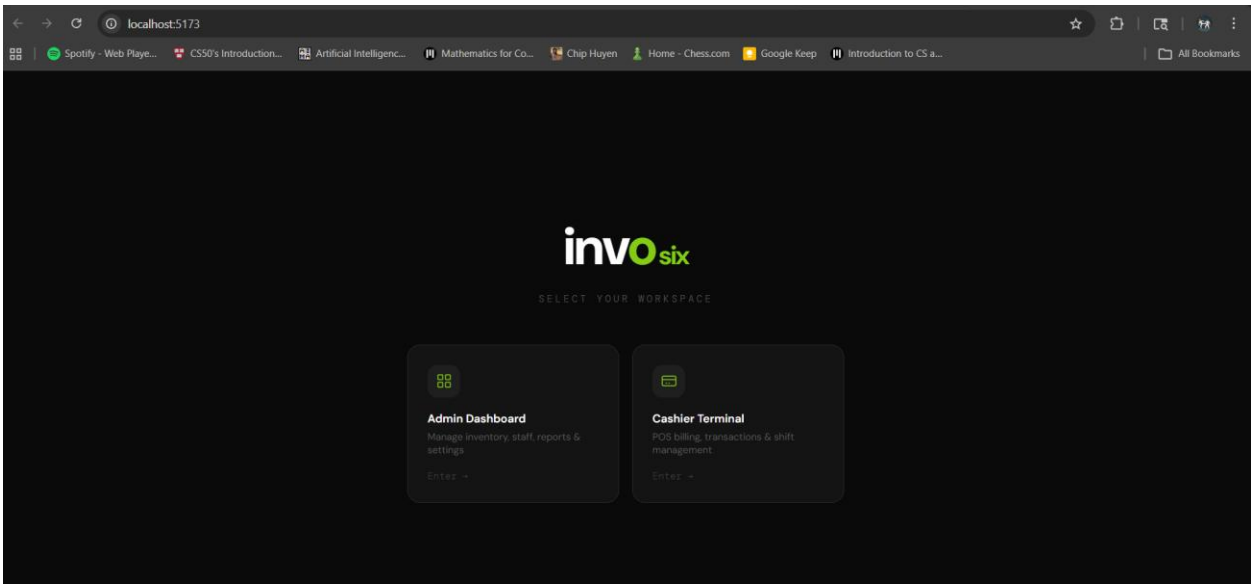

Fig: ERSIS login page in the browser

Two Terminals = Two Servers

You now have two terminal windows open: one running the backend (port 8000) and one running the frontend (port 5173). Both must be running for the application to work properly.

5. Running the Project

Once setup is complete, here's how to start the application every day. You don't need to run all the setup steps again, just start the servers.

5.1 Daily Startup Checklist

Every time you want to use ERSIS, follow these steps in order:

Start the Backend

1. Open VS Code and open the project folder.
2. Open a terminal.
3. Navigate to the backend folder and activate the virtual environment:

```
cd backend
.venv\Scripts\activate
```

4. Start the backend server:

```
python -m uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Start the Frontend

1. Open a second terminal.
2. Navigate to the frontend-web folder:

```
cd frontend-web
```

3. Start the frontend:

```
npm run dev
```

5.2 Accessing the Application

Service	Address
Web Dashboard (Admin/Cashier)	http://localhost:5173
API Documentation (Swagger)	http://127.0.0.1:8000/docs
MySQL Database (via Workbench)	localhost · Port 3306
Mobile App	Run via Expo (see Section 7)

5.3 What Success Looks Like

When everything is running correctly:

- The ERSIS login screen appears at http://localhost:5173
- You can log in with admin_seed@store.np / Password@123
- The dashboard loads with products, categories, and supplier data
- The Swagger page at http://127.0.0.1:8000/docs shows all available API endpoints

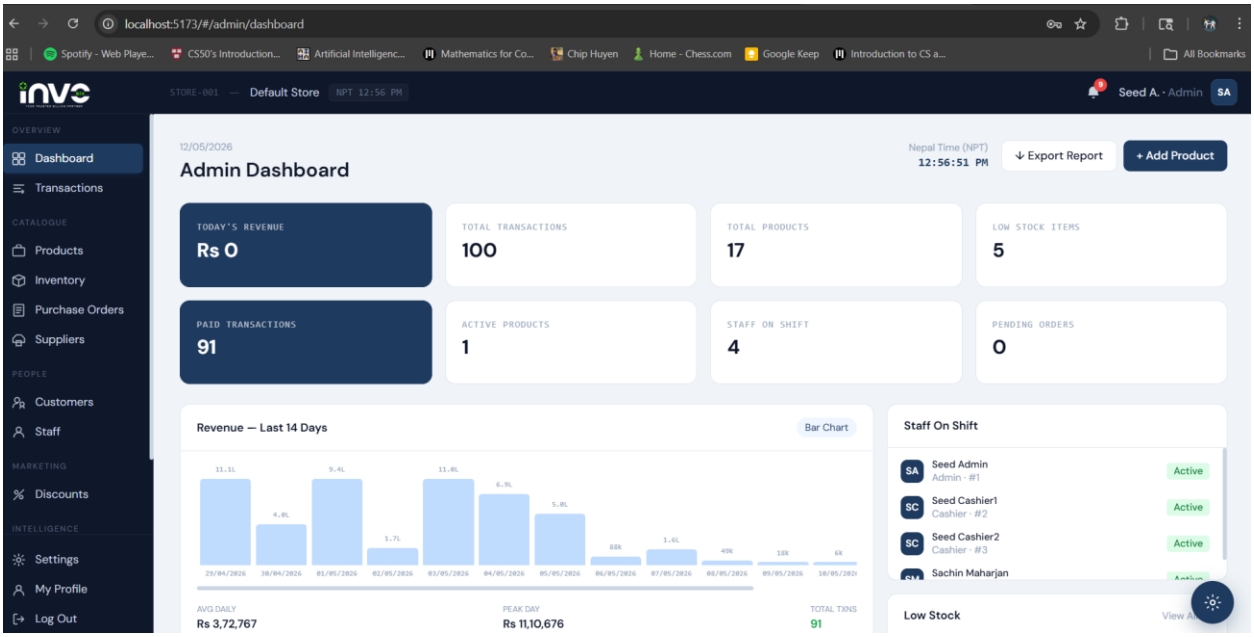

Fig: ERSIS Admin Dashboard after successful login

6. Mobile App Setup (React Native / Expo)

ERSIS includes a customer-facing mobile app for iOS and Android. It uses a tool called Expo, which lets you run the app on a real phone without needing to build a full Android or iOS package.

Backend Must Be Running First

The mobile app communicates with the backend. Make sure the backend server (Step 6 of Section 4) is already running before starting the mobile app.

6.1 Install Expo Go on Your Phone

Expo Go is a free app that lets you run Expo-based React Native projects directly on your phone. Install it before continuing.

- Android: Search 'Expo Go' on the Google Play Store
- iPhone: Search 'Expo Go' on the Apple App Store

6.2 Start the Mobile App

Open a new terminal in VS Code. Navigate to the mobile app folder:

```
cd frontend-mobile/myApp
```

Install dependencies (first time only):

```
npm install
```

Start the Expo development server:

```
npx expo start
```

A QR code will appear in your terminal. Your browser may also open to the Expo Dev Tools.

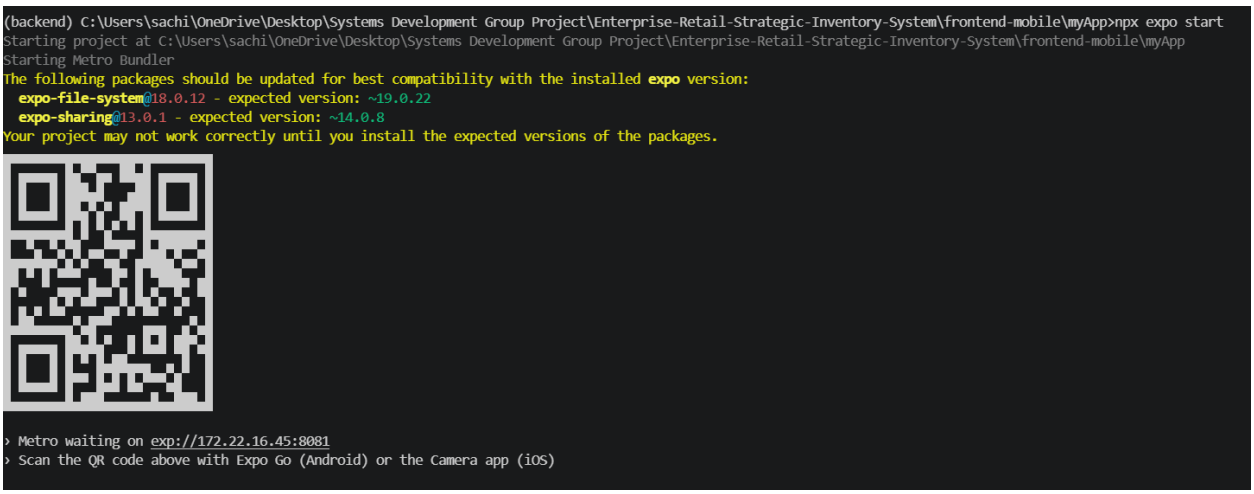

Fig: Terminal showing Expo Dev Tools with QR code

6.3 Opening the App on Your Device

Target Device	How to Open
Physical phone	Scan the QR code using the Expo Go app on your phone

Same Wi-Fi Required

Your phone and computer MUST be connected to the same Wi-Fi network.
Your computer's firewall must allow connections on Port 8000.
If the app cannot connect to the backend, see the Troubleshooting section (Section 8).

6.4 Connecting to the Backend

The mobile app automatically detects your computer's IP address. Watch the terminal for a line like:

```
[Config] Detected Backend URL: http://192.168.x.x:8000/api/v1
```

If autodetection fails, open the file frontend-mobile/myApp/src/constants/config.js in VS Code and manually set the IP address to your computer's local IP address.

Find Your Computer's IP

Windows: Open Command Prompt and type 'ipconfig'. Look for the IPv4 Address under your Wi-Fi adapter, e.g., 192.168.1.15

7. Docker Setup - Optional / Recommended

7.1 What is Docker?

Imagine Docker as a self-contained box that holds your entire application, the database, the backend server, and the web frontend, all pre-configured and ready to go. Instead of manually installing Python, Node.js, and MySQL, Docker handles everything automatically with a single command.

Benefits of using Docker for ERSIS:

- No need to install Python, Node.js, or MySQL separately
- Everything starts with one command: docker compose up
- Your computer's own software is not affected
- Perfect for sharing the project with others, it works the same on every computer

Docker vs Manual

The manual method gives you more control and is better for actively developing the code. Docker is better for quickly running the project to try it out, or for demonstrations.

7.2 Install Docker Desktop

If you haven't installed Docker yet, follow the instructions in Section 2.6. Make sure Docker Desktop is open and running (look for the whale icon in the system tray) before continuing.

7.3 Step-by-Step Docker Setup

Step 1 - Clone the Repository

If you haven't already downloaded the project, open a terminal and run:

```
git clone https://github.com/sachincode11/Enterprise-Retail-Strategic-Inventory-System.git
cd Enterprise-Retail-Strategic-Inventory-System
```

Step 2 - Create Environment Files

Docker needs two .env files, one for Docker Compose and one for the backend inside the container.

Create the root .env file:

```
copy .env.docker.example .env
```

Open .env in VS Code and fill in your values:

```
MYSQL_ROOT_PASSWORD=YourStrongPassword!
MYSQL_DATABASE=ersis
JWT_SECRET_KEY=change-me-to-a-very-long-random-secret
GROQ_API_KEY=gsk_...your_key_here...
SMTP_USER=your-email@gmail.com
SMTP_PASSWORD=your-gmail-app-password
EMAIL_FROM=your-email@gmail.com
```

Create the backend .env file:

```
copy backend\.env.example backend\.env
```

Open backend/.env and set the same GROQ_API_KEY, SMTP_* values, and JWT_SECRET_KEY.

DATABASE_URL in backend/.env

The DATABASE_URL in backend/.env still points to localhost — that is intentional. Docker Compose automatically overrides it to point to the MySQL container when running.

Step 3 — Build and Start All Services

Run this single command to build and start the entire application (backend, frontend, MySQL, MQTT broker):

```
docker compose up -d --build
```

What this does:

- Pulls official Docker images for MySQL, Node.js, and Python
- Builds and configures all services
- Starts everything in the background (-d means 'detached mode' it runs without blocking your terminal)

This takes 3–5 minutes on the first run. Be patient!

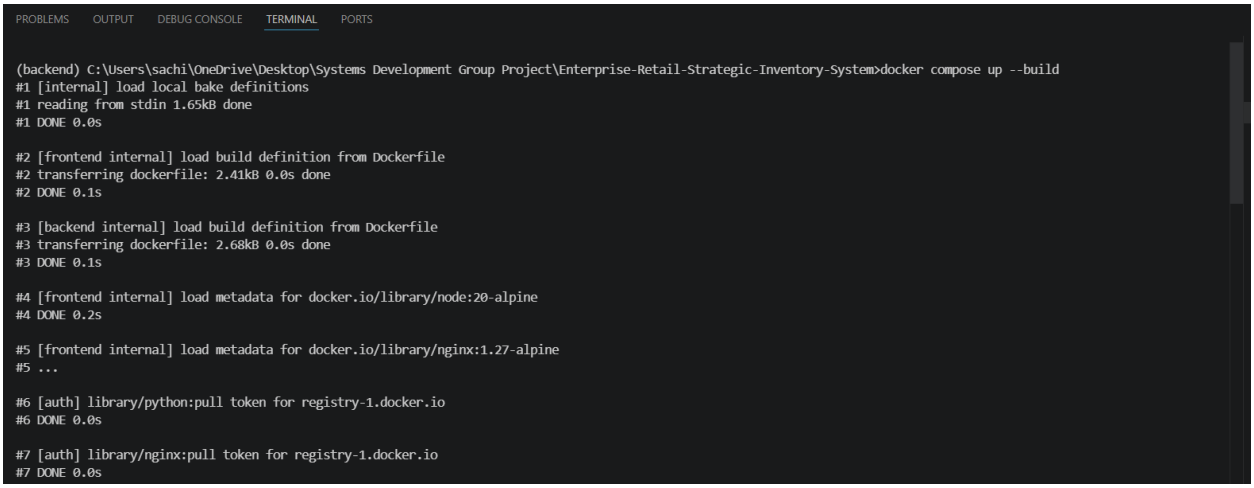

Fig: Terminal running 'docker compose up --build' with progress

Step 4 — Seed the Database

Once all containers are running, populate the database with sample data:

```
docker compose exec backend python seed.py
```

This inserts all test users, products, categories, transactions, and AI data.

Step 5 — Open the Application

Service	URL
Web App (Admin/Cashier Dashboard)	http://localhost
API Documentation (Swagger)	http://localhost:8000/docs
MySQL (connect via Workbench)	localhost : 3306
MQTT Broker (IoT devices)	localhost : 1883

7.4 Useful Docker Commands

Here are the essential Docker commands you'll use while working with ERSIS. Run them in the project root folder (where docker-compose.yml is located).

Command	What It Does
docker compose up -d --build	Build and start all services in the background
docker compose down	Stop all containers (keep your data)
docker compose down -v	Stop containers AND delete all data (fresh start — re-run seed.py after!)
docker compose logs -f backend	View live logs from the backend container
docker compose logs -f frontend	View live logs from the frontend container
docker compose exec backend bash	Open a command-line shell inside the backend container
docker compose up -d --build backend	Rebuild only the backend (e.g., after adding a Python package)

Warning: docker compose down -v

The -v flag deletes ALL stored data, including your database and AI indexes. After running this command, you must run 'docker compose exec backend python seed.py' again to restore your data.

7.5 Development Mode (Hot-Reload)

For active development, use the development override. This enables automatic code reloading, when you save a file, the server updates instantly:

```
docker compose -f docker-compose.yml -f docker-compose.dev.yml up --build
```

Service	URL in Dev Mode
Frontend (Vite HMR)	http://localhost:5173
Backend (Uvicorn reload)	http://localhost:8000
API Docs	http://localhost:8000/docs

8. Troubleshooting

Something not working? Don't panic — this section covers the most common problems beginners encounter and explains exactly how to fix them.

8.1 General & Manual Setup Issues

Problem	Solution
'python' is not recognized as a command	Python was not added to PATH during installation. Uninstall Python and re-install it, making sure to check 'Add Python to PATH' on the first screen.
'npm' is not recognized	Node.js was not installed correctly or the terminal is using an old session. Close and reopen the terminal or restart your computer.
'git' is not recognized	Git was not added to PATH. Restart your computer. If the problem persists, re-install Git and ensure 'Add to PATH' is selected.
'uv' command not found	Run 'pip install uv' in your terminal to install the uv package manager.
uv sync fails with lock error	Run 'uv lock' first to refresh the lock file, then try 'uv sync' again.
Backend starts but shows database errors	Check that MySQL is running and your DATABASE_URL in backend/.env has the correct password and database name.
Port 8000 already in use	Another program is using port 8000. Run: taskkill /F /PID <PID> (Windows) or change the port in the uvicorn command to --port 8001.
Port 5173 already in use	Stop the existing process or run npm run dev -- --port 5174 to use a different port.
npm install fails with EACCES (permission error)	Run the terminal as Administrator (right-click Command Prompt > Run as administrator).
seed.py fails with OperationalError	Make sure MySQL is running and you've created the 'ersis' database. Double-check the DATABASE_URL in backend/.env.
FAISS index not found error	Run 'python seed_ai.py' inside the backend folder (with virtual environment active).

8.2 Docker Issues

Problem	Solution
Docker not starting / whale icon missing	Restart Docker Desktop from the Start Menu. If it fails, restart your computer.
Backend container exits immediately	Run 'docker compose logs backend' to see the error. Usually, a missing or incorrect value in backend/.env.
MySQL keeps retrying for a long time	Normal on the first run — MySQL takes up to 60 seconds to initialize its data directory. Wait and watch the logs.
Blank page after docker compose up	Rebuild the frontend: docker compose up -d --build frontend

Hot-reload not working on Windows	Enable file sharing for the project drive: Docker Desktop → Settings → Resources → File Sharing
Port 80 already in use	Edit docker-compose.yml and change '80:80' to '8080:80'. Access the app at http://localhost:8080.
'Cannot connect to the Docker daemon'	Docker Desktop is not running. Open it from the Start Menu and wait for the whale icon to appear.

8.3 Mobile App Issues

Problem	Solution
QR code won't scan	Make sure your phone and computer are on the same Wi-Fi network. Try moving closer to your Wi-Fi router.
'Network request failed' on the phone	Replace 'localhost' in the app config with your computer's actual IP address (e.g., 192.168.1.15:8000). See config.js.
'expo: command not found'	Use 'npx expo start' instead of 'expo start'. Or install globally: npm install -g expo-cli
Metro bundler port conflict	Add --port 8082 to the command: npx expo start --port 8082
Expo Go shows 'Something went wrong'	Check that the backend server is running on port 8000. Verify the API URL in config.js matches your machine's IP address.

8.4 IoT Barcode Scanner Issues

Problem	Solution
ESP32 won't connect to Wi-Fi	Check WIFI_SSID and WIFI_PASSWORD in Config.h. Note: ESP32 only supports 2.4 GHz Wi-Fi, not 5 GHz.
HTTP error: connection refused	The backend is not running, or API_BASE_URL IP address is wrong. Use your machine's local network IP.
HTTP error: timeout	A firewall is blocking port 8000. Run the backend with --host 0.0.0.0 (already included in the start command).
403 Forbidden response	The IOT_DEVICE_SECRET in Config.h doesn't match the value in backend/.env.
POS badge shows 'No Device'	Wait 30–45 seconds after the ESP32 boots. If it persists, check the heartbeat in the PlatformIO Serial Monitor.

9. Frequently Asked Questions

Do I need programming knowledge to set up ERSIS?

No! This guide is designed for complete beginners. You will be copying and pasting commands from this guide. As long as you follow each step carefully and in order, you do not need to understand programming to get the system running.

What if an installation step fails?

Check the Troubleshooting section (Section 8) first. If you cannot find your specific issue, try searching the exact error message on Google. Most common errors have solutions on StackOverflow or GitHub Issues. You can also try restarting your computer and running the command again.

Can I use an IDE other than VS Code?

Yes. VS Code is recommended because it's free, lightweight, and has excellent extensions for Python and JavaScript. However, you can use WebStorm, PyCharm, Sublime Text, or any other editor. The terminal commands in this guide work in any terminal, not just VS Code's built-in one.

Why is Docker recommended if the manual method also works?

Docker eliminates the need to install Python, Node.js, and MySQL on your own computer. It also ensures that everyone who runs the project gets exactly the same environment, which prevents the famous 'it works on my machine' problem. For production deployments or sharing with a team, Docker is always the better choice.

Can I run ERSIS on a Mac or Linux computer?

Yes! The project runs on Windows, macOS, and Linux. Most commands are the same. Where commands differ (for example, 'copy' on Windows vs 'cp' on Mac/Linux), this guide shows both versions. For Docker, the setup is identical on all three platforms.

What is Groq AI and why do I need an API key?

Groq is a cloud-based AI service that powers ERSIS's AI assistant and chatbot features. You need a free Groq API key to enable these features. Sign up at console.groq.com — the free tier provides plenty of requests for development and testing.

What if I forget to activate the virtual environment?

If you run Python commands without activating the virtual environment, Python won't find the project's packages and will show 'ModuleNotFoundError'. Always activate the virtual environment before running backend commands. On Windows: .venv\Scripts\activate — look for (.venv) in your terminal prompt to confirm it's active.

How do I stop the application?

Press Ctrl+C in each terminal window (backend and frontend) to stop the servers. For Docker, run 'docker compose down' in the project folder. Your database data is preserved when you stop normally.

Is it safe to share my .env file?

No! The .env file contains passwords and secret keys. Never share it, commit it to Git, or upload it anywhere. The project is configured to ignore .env files in Git (.gitignore) to protect you, but be careful not to share them manually.

10. Final Setup Checklist

Use this checklist to confirm that everything is installed and working correctly. Tick off each item as you complete it.

Software Installation

☐	Git installed — git --version returns a version number
☐	Node.js 20 LTS installed — node --version returns v20.x.x
☐	Python 3.13 installed — python --version returns Python 3.13.x
☐	uv installed — uv --version returns a version number
☐	MySQL 8.0 installed and running — Can connect via MySQL Workbench or terminal
☐	Visual Studio Code installed
☐	Docker Desktop installed (optional) — Only if using Docker method

Project Configuration

☐	Project cloned or extracted to a local folder
☐	MySQL database 'ersis' created
☐	backend/.env file created and filled in — DATABASE_URL, GROQ_API_KEY, JWT_SECRET_KEY, SMTP settings
☐	Virtual environment created (uv venv) and activated
☐	Backend dependencies installed (uv sync)
☐	Database seeded (python seed.py) — done once

Servers Running

☐	Backend server started — Running at http://127.0.0.1:8000
☐	Swagger API docs accessible — http://127.0.0.1:8000/docs opens in browser
☐	Frontend dev server started — Running at http://localhost:5173
☐	ERSIS login page loads in browser
☐	Can log in with admin_seed@store.np / Password@123
☐	Admin dashboard loads with data (products, categories, etc.)

Mobile App (Optional)

☐	Expo Go app installed on phone
☐	npm install run in frontend-mobile/myApp
☐	npx expo start launches Expo Dev Tools
☐	QR code scanned - app opens on phone
☐	Mobile app connects to backend API

Docker Setup (Optional — Alternative to Manual)

☐	Docker Desktop installed and running
☐	Root .env file created from .env.docker.example
☐	backend/.env file created and filled in
☐	docker compose up -d --build completes without errors
☐	Database seeded via docker compose exec backend python seed.py
☐	Web app accessible at http://localhost
☐	Swagger docs accessible at http://localhost:8000/docs

Congratulations!
ERSIS is fully set up and ready to use. You can now manage inventory, process sales, run reports, and serve customers from a single unified system.